

1 Report3

1.1 inf 的旅行计划

1.1.1 形式化定义

有两个城市，分别有 n_1 和 n_2 个小镇，以及 m_1 和 m_2 条路径。每条路径连接同一城市中的两个小镇，经过路径需要消耗 x_i 的油费。同一城市的所有小镇通过路径连通。旅行者 INF 住在城市 1 的 1 号小镇，计划前往城市 2 的 1 号小镇。现需在两个城市中各选一个小镇建立一条新路径，消耗油费为 k 。由于无法预知具体选择的小镇，需计算保证到达目的地所需的最少油费。

输入格式 第一行包含五个正整数 n_1, n_2, m_1, m_2, k ，分别表示：

- 城市 1 的小镇数量 n_1 和路径数量 m_1 ，
- 城市 2 的小镇数量 n_2 和路径数量 m_2 ，
- 新建路径的油费 k 。

接下来 m_1 行，每行三个正整数 u_{i1}, v_{i1}, x_{i1} ，表示城市 1 中小镇 u_{i1} 和 v_{i1} 之间存在一条路径，油费为 x_{i1} 。

随后 m_2 行，每行三个正整数 u_{i2}, v_{i2}, x_{i2} ，表示城市 2 中小镇 u_{i2} 和 v_{i2} 之间存在一条路径，油费为 x_{i2} 。

输出格式 一行一个整数，表示保证到达城市 2 的 1 号小镇所需的最少油费。

1.1.2 解题思路

给定两个城市的图结构，我们需要在两个城市中各选一个节点建立一条新边，使得从城市 1 的起点到城市 2 的起点的路径油费最小化（在最坏情况下）。以下是解题步骤：

1. 图结构存储与预处理

- 使用邻接表存储两个城市的图结构（城市 1 和城市 2 分别用 `type=0` 和 `type=1` 区分）。
- 每条边是双向的，因此在输入时需添加正向和反向边。

2. 计算单源最短路径

- 使用 Dijkstra 算法分别计算：
 - 城市 1 中从起点 1 到所有其他节点的最短油费 `dist[0][i]`。
 - 城市 2 中从起点 1 到所有其他节点的最短油费 `dist[1][i]`。
- 由于所有边权均为正，Dijkstra 算法可以高效求解。

3. 分析最坏情况下的油费

- 新建边的油费为 k ，因此总油费为：

$$\text{Cost}(u, v) = \text{dist}[0][u] + k + \text{dist}[1][v]$$

- 由于无法预知具体选择的 u 和 v ，需考虑最坏情况，即：

$$\text{Answer} = \max_{u \in S_1, v \in S_2} (\text{dist}[0][u] + k + \text{dist}[1][v])$$

- 为了最小化该最大值，应选择使得 $\text{dist}[0][u]$ 和 $\text{dist}[1][v]$ 尽可能小的 u 和 v 。

4. 求解最小化最大油费

- 实际上，最坏情况发生在：
 - 城市 1 中选择离起点 1 最远的节点 u （即 $\text{dist}[0][u]$ 最大）。
 - 城市 2 中选择离起点 1 最远的节点 v （即 $\text{dist}[1][v]$ 最大）。
- 因此，最终答案为：

$$\text{Answer} = \max_{2 \leq i \leq n_1} \text{dist}[0][i] + k + \max_{2 \leq j \leq n_2} \text{dist}[1][j]$$

- 代码中通过遍历 $\text{dist}[0]$ 和 $\text{dist}[1]$ 数组找到最大值并相加得到结果。

1.1.3 复杂度分析

- Dijkstra 算法的时间复杂度为 $O(m \log n)$ ，其中 m 是边数， n 是节点数。
- 由于对两个城市分别运行一次 Dijkstra，总时间复杂度为 $O((m_1 + m_2) \log(n_1 + n_2))$ 。
- 后续遍历求最大值的时间复杂度为 $O(n_1 + n_2)$ ，可以忽略。

1.2 最小异或路径

1.2.1 形式化定义

给定一个简单无向连通图 $G = (V, E)$ ，其中：

- 节点集合 $V = \{1, 2, \dots, n\}$ ， $|V| = n$ ；
- 边集合 E ， $|E| = m$ ，每条边 $e = (u_i, v_i) \in E$ 具有边权 w_i 。

定义从节点 1 到节点 n 的简单路径 P 的异或和为：

$$\bigoplus_{(u,v) \in P} w_{(u,v)}$$

其中：

- 简单路径：路径不重复经过任何节点；
- 异或和 ($\oplus$)：路径上所有边权的按位异或结果。

目标 在所有可能的简单路径 P 中，找到使得异或和最小的路径，并输出该最小异或和：

$$\text{Answer} = \min_{P \in \mathcal{P}(1,n)} \left(\bigoplus_{(u,v) \in P} w_{(u,v)} \right)$$

其中 $\mathcal{P}(1,n)$ 表示所有从节点 1 到节点 n 的简单路径集合。

1.2.2 解题思路

给定一个简单无向连通图，要求找到从节点 1 到节点 n 的所有简单路径中边权异或和最小的路径。以下是基于深度优先搜索（DFS）的解题步骤：

1. 图表示与初始化

- 使用邻接表存储图结构，其中每个节点保存其相邻节点及对应边权。
- 初始化全局变量：
 - `min_xor`: 记录当前最小异或和，初始设为最大值（`INF`）。
 - `vis`: 标记节点是否被访问过的数组，初始为未访问（`false`）。

2. 深度优先搜索（DFS）

- 从起点 1 开始递归搜索，维护当前路径的异或和 `cur_xor`。
- 递归终止条件：到达目标节点 n 时，更新 `min_xor`：

$$\text{min_xor} = \min(\text{min_xor}, \text{cur_xor})$$

- 对于当前节点 u 的每个未访问邻居 v ：
 - 标记 v 为已访问。
 - 递归计算 v 到 n 的路径异或和：`cur_xor  $\oplus$   $w$` （ w 为边权）。
 - 回溯时恢复 v 的未访问状态，确保其他路径可探索。

3. 算法正确性

- **简单路径约束**：通过 `vis` 数组避免重复访问节点，保证路径简单性。
- **全局最优性**：DFS 遍历所有可能的简单路径，通过比较 `cur_xor` 确保找到最小异或和。
- **连通性保证**：题目给定的图连通，确保至少存在一条从 1 到 n 的路径。

1.2.3 复杂度分析

- 最坏情况下需遍历所有简单路径，时间复杂度为 $O((n-1)!)$ （如完全图）。
- 实际应用中，可通过剪枝优化（如提前终止大于当前 `min_xor` 的路径），但无法改变指数级复杂度。

1.3 树学长 QwQ 下象棋只会用车

1.3.1 形式化定义

在阶梯形地图中放置 k 个车，使得他们不会互相攻击的方案数汇总。

1.3.2 解题思路

1. 预处理阶乘和逆元：

- 使用快速幂算法预先计算阶乘数组 `fact` 和逆阶乘数组 `inv_fact`，模数为 100003。
- 逆元通过费马小定理计算： $a^{-1} \equiv a^{p-2} \pmod{p}$ 。

2. 组合数计算：

- 函数 $C(n, m)$ 利用预处理的阶乘和逆元计算组合数 $\binom{n}{m} \pmod{100003}$ 。
- 处理边界情况：当 $m > n$ 或 $m < 0$ 时返回 1。

3. 主计算逻辑：

- 遍历所有可能的分配方式：将 k 分为 i （分配给 c 和 d 类）和 $j = k - i$ （分配给其他类）。
- 约束条件：
 - $i \leq \min(c, d, k)$
 - $j = k - i \geq 0$
 - $a + e \geq j$ （确保有足够容量）
- 对每个有效分配计算组合数乘积：

$$\binom{b+d-i}{j} \times \binom{a+e}{j} \times j! \times \binom{c}{i} \times \binom{d}{i} \times i!$$

- 将各项相乘并对结果取模。

4. 输出结果：

- 累加所有有效分配方式的结果，输出最终值 $\text{mod} 100003$ 。

1.3.3 复杂度分析

时间复杂度约为 $O(MAXN)$

1.4 Sherway 的卡牌

1.4.1 形式化定义

给定初始状态 $S_0 = \{(1)\}$ （仅含一张攻击力为 1 的卡牌）和事件序列 $A = (a_1, a_2, \dots, a_n)$ ，其中 $\forall a_i \in \{-1, 0, 1\}$ 。定义状态转移规则如下：

- 拿牌事件 ($a_i = 1$):

若当前状态为 $S_{i-1} = \{x_1, x_2, \dots, x_m\}$ ，则新状态为 $S_i = S_{i-1} \cup \{1\}$ ，其中 $m \geq 0$ 。

- 弃牌事件 ($a_i = -1$):
若 $|S_{i-1}| \geq 2$, 选择任意 $x_u, x_v \in S_{i-1}$, 转移至 $S_i = (S_{i-1} \setminus \{x_u, x_v\}) \cup \{x_u + x_v\}$; 否则游戏失败 (输出 -1)。
- 抽奖事件 ($a_i = 0$):
可选择执行 $a_i = 1$ 或 $a_i = -1$ 的转移规则 (必须选择其一)。

定义最终平均攻击力为:

$$\text{Avg}(S_n) = \frac{1}{|S_n|} \sum_{x \in S_n} x$$

输入:

第一行为 $n \in \mathbb{Z}^+$, 第二行为序列 $A = (a_1, \dots, a_n)$ 。

输出:

若存在转移路径使得 $\forall i \leq n, |S_i| \geq 1$, 则输出 $\max \text{Avg}(S_n)$ 的最简分数形式 $\frac{p}{q}$ (需满足 $\gcd(p, q) = 1$); 否则输出 -1。

1.4.2 解题思路

通过动态维护卡牌数量和攻击力总和, 结合贪心策略确保事件序列可行并最大化平均攻击力。

1. 预处理阶段 (逆向扫描):

- 定义数组 `req_min`, 其中 `req_min[i]` 表示处理完第 i 个事件后所需的最小卡牌数
- 初始化 `req_min[n] = 1` (最终至少保留 1 张牌)
- 逆向遍历事件序列, 根据事件类型更新最小需求:
 - 弃牌事件 ($a_i = -1$): 至少需要 $\max(2, \text{req_min}[i+1] + 1)$ 张牌
 - 拿牌事件 ($a_i = 1$): 至少需要 $\max(1, \text{req_min}[i+1] - 1)$ 张牌
 - 抽奖事件 ($a_i = 0$): 取拿牌和弃牌需求的最小值

2. 可行性判断:

- 若初始需求 `req_min[0] > 1`, 说明无法完成所有事件, 直接输出 -1

3. 正向处理阶段:

- 初始化当前卡牌数 $k = 1$ 和攻击力总和 $S = 1$
- 遍历事件序列动态更新状态:
 - 拿牌事件: k 和 S 均增加 1
 - 弃牌事件: k 减少 1 (攻击力总和不变)
 - 抽奖事件: 选择使 k 尽可能小的操作 (贪心策略):
 - * 若 $k - 1 \geq \text{req_min}[i+1]$ 则弃牌
 - * 否则拿牌

4. 结果计算:

- 计算 S 与 k 的最大公约数 $g = \gcd(S, k)$
- 输出最简分数形式 $\frac{S/g}{k/g}$

1.4.3 复杂度分析

时间复杂度 $O(n)$ （预处理和正向处理各扫描一次）